

California Housing ML Project

Complete 44-Day Master Timeline

From empty folder to a live, professional ML project
with CI/CD pipeline deployed on AWS

Total Duration	44 days (~6.5 weeks)
Daily Commitment	1–2 hours/day
Total Phases	13 phases
Final Output	Live ML web app on AWS with CI/CD
Skill Level	Beginner → Intermediate

Phase Overview

Phase	Days	Focus
1. Setup	1	Project structure & GitHub
2. Data Exploration	2–4	Understand the dataset (EDA)
3. Data Preparation	5–7	Clean, engineer, scale features
4. Modeling	8–12	Train, compare, tune models
5. Evaluation	13–15	Detailed analysis & SHAP
6. Web App	16–19	Build Streamlit app
7. First Deployment	20–21	Deploy to Streamlit Cloud

8. Documentation	22–23	Professional README
9. Docker	24–27	Containerize the app
10. Testing	28–30	pytest test suite
11. CI/CD	31–33	GitHub Actions pipeline
12. AWS	34–40	Deploy to production cloud
13. Monitoring	41–44	Logs, alerts, dashboards

PHASE 1 — Project Setup (Day 1)

Goal: Get your environment ready and plan the project before writing any ML code.

Day 1 — Setup & Foundations

- Install Python (or just use Google Colab — recommended for beginners)
- Install VS Code (free code editor) — you'll need it for non-notebook files later
- Install Git on your computer (or use GitHub Desktop)
- Create a GitHub account if you don't have one
- Create a new GitHub repository: **california-housing-regression**
- Clone the repo to your computer
- Create folder structure: `data/`, `notebooks/`, `src/`, `models/`, `app/`, `tests/`
- Create **PLAN.md** — one paragraph: what you're building, dataset, goal, deliverables
- First commit: `git add . && git commit -m 'Initial project setup'`
- Push to GitHub

End of day: Empty but organized project, linked to GitHub.

PHASE 2 — Data Exploration (Days 2–4)

Goal: Know your data inside out before training anything.

Day 2 — Load and Explore

- Create notebook **notebooks/01_data_exploration.ipynb**
- Import pandas, numpy, matplotlib, seaborn
- Load California Housing dataset from sklearn (`fetch_california_housing`)
- Convert to a DataFrame, add the Price column
- Run `.head()`, `.info()`, `.describe()`, `.shape`, `.isnull().sum()`
- Write markdown notes about what you see

End of day: You know your data's shape, columns, and quality.

Day 3 — Visualize Features

- Histogram of Price (target)
- Histograms of all features in one call
- Box plots to find outliers
- Correlation heatmap with seaborn
- Scatter plot: MedInc vs Price
- Geographic plot: Latitude vs Longitude, colored by Price

End of day: You've SEEN patterns in your data with at least 6 plots.

Day 4 — Data Analysis & Findings

- Identify the top 3 features most correlated with Price
- Spot data quality issues (the \$500K price cap, outliers in AveRooms)
- Write a 'Data Findings' markdown section: what you noticed, what may cause problems, which features should matter most
- Commit and push to GitHub

End of day: You can explain your dataset to a stranger.

PHASE 3 — Data Preparation (Days 5–7)

Goal: Get the data ready for modeling — the most overlooked but most important phase.

Day 5 — Handle Data Issues

- New notebook: **notebooks/02_data_preparation.ipynb**
- Check for duplicates: `df.duplicated().sum()`
- Decide what to do with the \$500K cap (document your choice in markdown)
- Investigate outliers in AveRooms and AveOccup
- Practice handling missing values (show the code even though there are none)

End of day: Data is clean and your decisions are documented.

Day 6 — Feature Engineering

- Create **RoomsPerHousehold** = AveRooms / AveOccup
- Create **BedroomsPerRoom** = AveBedrms / AveRooms
- Create **PopulationPerHousehold** = Population / AveOccup
- Brainstorm one feature of your own (try Distance_from_coast)
- Plot each new feature against Price to verify they help

End of day: Your dataset now has 11+ features including engineered ones.

Day 7 — Scaling & Save Prepared Data

- Split into X (features) and y (target)
- Train/test split (80/20, random_state=42)
- Apply StandardScaler — fit on train only, transform both
- Save prepared arrays with joblib (X_train.pkl, etc.)
- Save the scaler too (scaler.pkl)
- Commit and push

End of day: Data is ready for modeling.

PHASE 4 — Modeling (Days 8–12)

Goal: Build, compare, and tune multiple regression models.

Day 8 — Baseline: Linear Regression

- New notebook: **notebooks/03_modeling.ipynb**
- Load the prepared data
- Train a LinearRegression model
- Calculate MAE, RMSE, R^2
- Plot predicted vs actual
- Write what each metric means in your own words

End of day: Baseline numbers established (expect $R^2 \approx 0.60$).

Day 9 — Try More Models

- Train and compare: Ridge, Lasso, Decision Tree Regressor, Random Forest Regressor, Gradient Boosting Regressor
- Make a comparison table of MAE, RMSE, R^2
- Identify your top 2 performers

End of day: You can see which algorithms win on this data.

Day 10 — Cross-Validation

- Use `cross_val_score` with `cv=5` on your top 2 models
- Compare with simple train/test split scores
- Check if performance is stable across folds
- Document findings in markdown

End of day: You trust your numbers more.

Day 11 — Hyperparameter Tuning

- Pick your best model (likely Random Forest or Gradient Boosting)
- Set up GridSearchCV with parameters like `n_estimators`, `max_depth`, `min_samples_split`
- Run it (takes 10–30 minutes — be patient)
- Document the best parameters found

End of day: You have an optimized model.

Day 12 — Final Model & Save

- Train the final tuned model on the full training set
- Evaluate on the test set one final time
- Save model: `joblib.dump(model, 'models/final_model.pkl')`
- Write 'Model Report' markdown with final metrics and best params
- Commit and push

End of day: Trained model lives in `models/` folder.

PHASE 5 — Evaluation & Interpretation (Days 13–15)

Goal: Don't just have a model — understand what it does.

Day 13 — Detailed Evaluation

- Predicted vs actual scatter plot
- Residual plot (errors)
- Distribution of residuals (should be roughly normal)
- Find your worst 10 predictions — investigate why
- Bucket predictions by price range — where does the model struggle?

End of day: You know where your model wins and loses.

Day 14 — Feature Importance & SHAP

- Plot feature importances from Random Forest
- Install SHAP: `pip install shap`
- Generate SHAP summary plot
- Generate SHAP force plot for one specific prediction
- Write down what you learned

End of day: You can explain why your model makes its predictions.

Day 15 — Insights Document

- Write **INSIGHTS.md** with: top 3 things you learned about California housing, top 3 weaknesses of your model, what you'd do with more time/data
- Commit and push

End of day: You have written, polished insights.

PHASE 6 — Web App with Streamlit (Days 16–19)

Goal: Turn your model into something a real person can use.

Day 16 — Streamlit Basics

- Install Streamlit: `pip install streamlit`
- Create **app/app.py**
- Build a 'Hello World' Streamlit app
- Run it: `streamlit run app/app.py`
- Add title, description, basic input widgets (sliders)

End of day: A basic Streamlit page runs on your laptop.

Day 17 — Connect Model to App

- Load the saved model and scaler with `joblib.load()`
- Add input widgets for all 8 original features
- Add a 'Predict' button
- On click: scale inputs → predict → display result formatted as dollars

End of day: App predicts house prices from user input.

Day 18 — Polish the App

- Add header with project description
- Add sidebar with feature descriptions
- Add a map showing the predicted location (Latitude/Longitude)
- Show feature importance chart in the app
- Add example preset buttons ('San Francisco', 'Rural area')
- Add a 'How does this work?' section

End of day: Your app looks professional, not like a school project.

Day 19 — Test the App

- Try edge cases (zero rooms, negative income) — fix any crashes
- Add input validation
- Test on phone (Streamlit is mobile-friendly)
- Fix bugs
- Take a screenshot for the README
- Commit and push

End of day: App works reliably.

PHASE 7 — First Deployment (Days 20–21)

Goal: Put your app on the internet, for free.

Day 20 — Prepare for Deployment

- Create **requirements.txt** with all libraries used
- Create **.gitignore** (ignore `__pycache__`, `.ipynb_checkpoints`, etc.)
- Test in a fresh environment: `pip install -r requirements.txt && streamlit run app/app.py`
- Make sure everything works from scratch

End of day: Project is portable.

Day 21 — Deploy to Streamlit Cloud

- Push final code to GitHub
- Go to **share.streamlit.io** and sign in with GitHub
- Click 'New App' → select repo → set main file as `app/app.py`
- Deploy (takes ~5 minutes)
- You now have a **live URL** anyone can visit
- Share it with one friend

End of day: ■ Your app is LIVE on the internet.

PHASE 8 — Documentation (Days 22–23)

Goal: Make your GitHub repo look professional.

Day 22 — Write the README

- Write professional README.md with: Title + app screenshot, Live Demo link, Description, Dataset info, Methodology, Results table, Insights, Tech Stack, How to run locally, Project structure, What I learned
- Add badges from shields.io

End of day: README looks professional.

Day 23 — Final Polish

- Clean up notebooks (remove dead cells, make sure they run top to bottom)
- Add docstrings to any Python files
- Save plot images to images/ folder and embed in README
- Add a LICENSE file (MIT)
- Final commit and push

End of day: Repo looks like a real engineer made it.

■ MILESTONE — Day 23: Phase 1 Complete

You now have: A working ML project, a live Streamlit app, a polished GitHub repo, and real ML skills. This alone is portfolio-worthy. Many people stop here — and that's fine. If you want CI/CD and AWS, keep going to Phase 9.

PHASE 9 — Docker (Days 24–27)

Goal: Package your app into a portable container.

Day 24 — Learn Docker Basics

- Install Docker Desktop
- Understand: what is a container vs an image
- Run a test container: `docker run hello-world`
- Read about Dockerfile syntax (just read, no writing yet)

End of day: You understand what Docker is.

Day 25 — Write Your Dockerfile

- Create a file called **Dockerfile** in project root
- Use a Python base image
- Copy requirements.txt and install dependencies
- Copy app code
- Expose port 8501
- Set the streamlit command as entrypoint

End of day: Dockerfile exists.

Day 26 — Build and Run the Container

- Build image: `docker build -t california-housing .`
- Run container: `docker run -p 8501:8501 california-housing`
- Visit `localhost:8501` — your app runs inside Docker
- Debug any errors

End of day: Your app runs in a container on your machine.

Day 27 — Push to Docker Hub

- Create a Docker Hub account
- Tag your image properly
- Push image to Docker Hub
- Verify it appears on Docker Hub
- Commit and push the Dockerfile

End of day: Your app is containerized and shared.

PHASE 10 — Testing (Days 28–30)

Goal: Write automated tests so CI/CD has something to run.

Day 28 — Learn pytest Basics

- Install pytest: `pip install pytest`
- Create `tests/` folder with `test_basic.py`
- Write one simple passing test
- Run pytest — see green checkmarks

End of day: Testing framework is set up.

Day 29 — Write Tests for Your Model

- Create `tests/test_model.py`
- Test: model loads without errors
- Test: returns a number for valid input
- Test: returns reasonable values (not negative)
- Test: handles invalid input gracefully

End of day: Real tests covering your model.

Day 30 — Tests for App Logic

- Create `tests/test_app.py`
- Test the prediction function
- Test input scaling
- Test edge cases
- Run full suite: `pytest -v`
- Commit and push

End of day: Comprehensive test coverage.

PHASE 11 — CI/CD with GitHub Actions (Days 31–33)

Goal: A robot that automatically tests and builds your project on every push.

Day 31 — Learn GitHub Actions Basics

- Create folder `.github/workflows/`
- Create simple `hello.yml` workflow that just prints 'Hello'
- Push to GitHub
- Watch it run in the Actions tab

End of day: Your first automation runs.

Day 32 — Build the CI Pipeline

- Create `.github/workflows/ci.yml`
- On every push: check out code, set up Python, install requirements, run pytest
- Push code and watch pipeline run
- If tests fail → pipeline fails → fix and push again

End of day: Automated testing on every push.

Day 33 — Add Docker Build to Pipeline

- Add steps to CI workflow: build Docker image, push to Docker Hub
- Set up Docker Hub credentials as GitHub Secrets
- Push code and verify image is built and pushed automatically

End of day: Full CI pipeline working.

PHASE 12 — AWS Deployment (Days 34–40)

Goal: Deploy your app on Amazon Web Services like a professional.

■ **IMPORTANT:** The AWS free tier covers your usage IF you stay within limits. On Day 34, immediately set up billing alerts at \$1, \$5, and \$10 so you never get surprised by a bill. Always stop EC2 instances when you're done experimenting.

Day 34 — AWS Setup

- Create an AWS account (credit card needed)
- **Immediately set up billing alerts** — Billing → Budget alert at \$1, \$5, \$10
- Install AWS CLI on your laptop
- Configure with `aws configure`

End of day: AWS account ready and safe from accidental charges.

Day 35 — Launch EC2 Instance

- AWS Console → EC2 → Launch a **t2.micro** instance (free tier)
- Choose Ubuntu 22.04
- Create a key pair, download the .pem file (keep it safe!)
- Configure security group: allow SSH (port 22) and port 8501
- Note your instance's public IP

End of day: A virtual computer in the cloud, ready to use.

Day 36 — Connect to EC2 & Install Docker

- SSH into EC2: `ssh -i your-key.pem ubuntu@<public-ip>`
- Update system: `sudo apt update && sudo apt upgrade -y`
- Install Docker on EC2
- Test: `docker run hello-world`

End of day: Docker runs on your AWS server.

Day 37 — Run Your App on EC2

- Pull your image: `docker pull yourusername/california-housing`
- Run it: `docker run -d -p 8501:8501 yourusername/california-housing`
- Visit `http://<your-ec2-ip>:8501`
- Your app is live on AWS ■

End of day: App running on AWS, not Streamlit Cloud anymore.

Day 38 — Domain & HTTPS (optional but recommended)

- Buy a cheap domain (\$5–10) or use a free subdomain
- Point it to your EC2's IP
- Set up Nginx as a reverse proxy
- Get a free SSL certificate from Let's Encrypt

End of day: App available at `https://yourdomain.com` instead of an IP.

Day 39 — Connect CI/CD to AWS

- Add a new step to your GitHub Actions workflow: SSH into EC2 → pull new image → restart container
- Set up SSH keys as GitHub Secrets
- Push a code change and watch the full pipeline run end-to-end

End of day: Full end-to-end automation. Push code → live in 5 minutes.

Day 40 — Cleanup & Cost Check

- Check AWS billing — should be \$0 or very low
- Verify billing alerts are still active
- Clean up any unused resources
- Document AWS setup in **DEPLOYMENT.md**

End of day: Deployment documented, costs controlled.

PHASE 13 — Monitoring (Days 41–44)

Goal: Know if your app is healthy and get notified when it isn't.

Day 41 — Add Logging

- Add Python's logging module to your Streamlit app
- Log every prediction with timestamp and inputs
- Log errors with full traceback
- Test by triggering errors intentionally

End of day: Your app generates useful logs.

Day 42 — CloudWatch Setup

- AWS Console → CloudWatch
- Send EC2 logs to CloudWatch
- Create a dashboard showing: predictions/hour, average response time, error rate

End of day: You can see your app's health from anywhere.

Day 43 — Alerts

- Set up CloudWatch alarms: email if error rate > 5%, email if EC2 CPU > 80%, email if no predictions in 24 hours
- Test by triggering an alarm

End of day: You'll know when something breaks.

Day 44 — Final Wrap-up & Reflection

- Update README with full architecture diagram (use draw.io or excalidraw)
- Add CI/CD badge to README (shows pipeline status)
- Update **INSIGHTS.md** with everything learned in Phases 9–13
- Write **REFLECTION.md** for yourself: hardest part, most valuable lesson, what to build next
- Final commit

End of day: ■ Done. Full enterprise-grade ML project complete.

■ The Final Outcome

Deliverable	Status
ML model trained, tuned, evaluated	✓
Web app users can interact with	✓
Docker container (portable anywhere)	✓
Automated tests	✓
CI/CD pipeline (push code → auto-deploy)	✓

Live on AWS with your own domain	✓
Monitoring & alerts	✓
Professional documentation	✓
Real understanding of every piece	✓

■ Critical Rules for All 44 Days

- **One day at a time.** Don't read Day 30 today. Read Day 1.
- **It's okay to take longer.** Day 11 might take you 2 days — that's fine. The plan adjusts to you.
- **Type code, don't paste.** Even from examples. Typing equals learning.
- **Markdown explanations everywhere.** Every notebook section needs 'what I did and why' in plain English.
- **Commit and push every 1–2 days minimum.** Don't lose work. Don't make giant commits.
- **Stuck? Ask concept questions only.** Not 'write this for me.'
- **Rest is part of learning.** Take a day off when burnt out. The plan will wait.
- **Don't add side projects.** No LLMs, no NLP 'while' doing this. Finish first.

Your Next Action Right Now

Stop reading. Stop planning. Open Google Colab. Create the project folder. Start Day 1, Task 1. You have the full map now — the only thing missing is the walking. Go.